

PORTABLE ESP32-BASED AIR QUALITY MONITORING SYSTEM

Project Developed By

Project Developer: Guntur Anurag, Ayesha Routray

Personal Technical Development Project

Independent / Self-Development Work

Project Period: 2026–2027

ABSTRACT

Air pollution monitoring requires timely information about particulate matter and surrounding environmental conditions. This project presents a portable, battery-powered air-quality monitoring prototype based on the ESP32 microcontroller. The system combines a BME680 environmental sensor, PMS7003 particulate sensor, SH1106 128×64 OLED display, rechargeable LiPo battery, TP4056 charging/protection circuit, and MT3608 boost converter.

The BME680 measures temperature, humidity, pressure, and gas resistance, while the PMS7003 measures PM1.0, PM2.5, and PM10. The ESP32 acts as the central processing and communication unit. PM2.5 and PM10 measurements are converted into pollutant-specific AQI sub-indices using linear interpolation between configured breakpoints, and the higher valid sub-index is used as the final AQI.

The system provides local readings through the OLED. Wireless connectivity is implemented through ESPHome, allowing integration with Home Assistant for a detailed dashboard. A built-in ESPHome web server provides a lightweight portable interface that can be accessed from a phone when the ESP32 is connected to a suitable Wi-Fi network. A fallback access point and captive portal support network reconfiguration.

The prototype successfully demonstrated sensor communication, local display, particulate monitoring, AQI processing, Home Assistant integration, and portable Wi-Fi operation. The development also identified power stability and hotspot compatibility as areas requiring further optimization.

TABLE OF CONTENTS

1.	Introduction
2.	Problem Statement, Aim and Objectives
3.	System Requirements and Scope
4.	Technical Background
5.	System Architecture and Design
6.	Hardware Implementation
7.	Software and Firmware Implementation
8.	AQI Calculation and Data Processing
9.	User Interfaces and IoT Connectivity
10.	Methodology and Prototype Development
11.	Testing and Validation
12.	Troubleshooting and Fault Analysis
13.	Results and Discussion
14.	Advantages, Limitations and Applications
15.	Future Scope
16.	Conclusion and References

LIST OF TABLES, FIGURES AND ABBREVIATIONS

LIST OF TABLES

- Table 1. Hardware components and functions
- Table 2. Pin configuration
- Table 3. Communication interfaces
- Table 4. Power-system components
- Table 5. AQI breakpoint ranges
- Table 6. AQI status categories
- Table 7. Testing and validation results
- Table 8. Troubleshooting observations
- Table 9. Advantages and limitations
- Table 10. Future enhancement areas

LIST OF FIGURES

- Figure 1. Overall system architecture
- Figure 2. Sensor-to-ESP32 data flow
- Figure 3. Power supply architecture
- Figure 4. Home Assistant monitoring architecture
- Figure 5. Portable ESPHome web architecture
- Figure 6. AQI processing flow

ABBREVIATIONS

Abbreviation	Meaning
AQI	Air Quality Index
IoT	Internet of Things
I ² C	Inter-Integrated Circuit
UART	Universal Asynchronous Receiver/Transmitter
OTA	Over-The-Air
PM	Particulate Matter
OLED	Organic Light Emitting Diode
LiPo	Lithium Polymer
GPIO	General Purpose Input/Output
AP	Access Point
ESP-IDF	Espressif IoT Development Framework

1. INTRODUCTION

Air-quality monitoring is an important application of embedded systems and the Internet of Things (IoT), particularly for continuous observation of temperature, humidity, and particulate pollution. Conventional air-quality monitoring stations are generally large and fixed in location, whereas a portable monitoring system can provide measurements directly at the location of interest. The objective of this project is to develop a compact and portable air-quality monitoring system capable of sensing environmental conditions, processing the acquired data using an ESP32 microcontroller, and presenting the results in an easily understandable form.

The proposed system is built around the **ESP32**, which acts as the central processing and communication unit. It receives measurements from the connected sensors, processes the data, calculates the corresponding **Air Quality Index (AQI)**, and provides the results to the user. Its integrated Wi-Fi connectivity and compact size make it suitable for portable embedded applications.

The system uses the **BME680** for measuring temperature and relative humidity, while the **PMS7003** measures airborne particulate matter, particularly **PM2.5 and PM10**. PM2.5 refers to fine particles with an aerodynamic diameter of 2.5 μm or smaller, while PM10 refers to particles with an aerodynamic diameter of 10 μm or smaller. These measurements are used to evaluate the particulate pollution level of the surrounding environment.

The PM2.5 and PM10 values are processed independently to obtain their respective **AQI sub-indices**, from which the overall AQI is determined using the implemented AQI calculation method. The system therefore provides both individual particulate measurements and a single AQI value representing the overall air-quality condition.

A **128 × 64 OLED display** provides a local, real-time interface for displaying temperature, humidity, PM2.5, PM10, AQI, and AQI status. The prototype is powered by a **Li-Po battery**, with a **TP4056 charging module** and **MT3608 boost converter** forming the power-management section.

The ESP32 also provides Wi-Fi connectivity through **ESPHome**, allowing the sensor readings to be accessed through a web interface from a device connected to the same local network. Thus, the prototype supports both local monitoring through the OLED display and wireless monitoring through a connected device.

The overall architecture of the prototype can be represented as:

Sensors → ESP32 → Data Processing & AQI Calculation → OLED Display / Wi-Fi Web Interface

The project combines environmental sensing, embedded processing, AQI computation, portable power, and wireless monitoring into a compact and practical air-quality monitoring system.

2. PROBLEM STATEMENT, AIM AND OBJECTIVES

Problem Statement

Conventional air-quality monitoring systems are often fixed and unsuitable for quick localized measurements. A compact and portable system is therefore required for real-time monitoring of environmental conditions and particulate pollution.

Aim

To design and develop a **portable ESP32-based air-quality monitoring system** that measures environmental parameters and particulate matter, calculates AQI, displays the results locally, and provides wireless monitoring.

Objectives

1. Measure **temperature and humidity** using the BME680.
2. Measure **PM1.0, PM2.5, and PM10** using the PMS7003.
3. Calculate **PM2.5 and PM10 AQI sub-indices** and determine the final AQI.
4. Display sensor readings, AQI, and AQI status on the **OLED display**.
5. Provide wireless monitoring through **Wi-Fi and ESPHome**.
6. Integrate the system with **Home Assistant**.
7. Develop a **battery-powered perfboard prototype** using a rechargeable Li-Po battery.
8. Test and validate the integrated hardware and software system.

Project Scope

The project covers **environmental sensing, AQI calculation, OLED display, Wi-Fi communication, ESPHome/Home Assistant integration, and portable battery operation**. The prototype is intended for demonstration and educational purposes and is not designed as a laboratory-grade or regulatory-certified air-quality instrument.

Personal Development Outcomes

The project provided practical experience in **ESP32 development, I²C and UART communication, sensor integration, AQI processing, wireless communication, power management, ESPHome, Home Assistant, and hardware troubleshooting**.

The prototype was developed through **hardware assembly, electrical testing, firmware configuration, sensor integration, and system validation**.

3. SYSTEM REQUIREMENTS AND SCOPE

Hardware Requirements

Component	Role
ESP32	Main controller, processing and Wi-Fi
BME680	Temperature, humidity, pressure and gas resistance
PMS7003	PM1.0, PM2.5 and PM10
SH1106 OLED	Local live display
3.7 V 2200 mAh LiPo	Rechargeable power source
TP4056	Charging/protection
MT3608	Boost conversion to regulated supply
Perfboard and headers	Prototype assembly and modular mounting
Slide switch	Intended ON/OFF control

Software Requirements

Software	Purpose
ESPHome	Firmware configuration and device integration
ESP-IDF	ESP32 firmware framework
Home Assistant	IoT dashboard and visualization
Web browser	Portable ESPHome interface
Serial/log output	Debugging and validation

Functional Requirements

Acquire sensor measurements

- Process particulate data
- Calculate AQI
- Display local values
- Connect to Wi-Fi
- Publish values to Home Assistant
- Host a portable web interface
- Support fallback Wi-Fi configuration
- Support OTA firmware updates

4. TECHNICAL BACKGROUND

ESP32

The ESP32 is the main controller of the prototype. It collects data from the BME680 and PMS7003, processes the sensor readings, calculates the AQI, and controls the OLED display. It also provides Wi-Fi connectivity for ESPHome and Home Assistant monitoring.

BME680

The BME680 is used to measure temperature, humidity, pressure, and gas resistance. It communicates with the ESP32 using the I²C protocol. In the current system, temperature and humidity are used for environmental monitoring, while the particulate AQI is calculated separately from the PMS7003 readings.

PMS7003

The PMS7003 is the main particulate-matter sensor. It measures PM1.0, PM2.5, and PM10 and communicates with the ESP32 through UART. The PM2.5 and PM10 values are used to calculate their respective AQI sub-indices and the final AQI.

OLED Display

A 128 × 64 SH1106 OLED is used to display the sensor readings locally. It communicates with the ESP32 through I²C and displays parameters such as temperature, humidity, PM2.5, PM10, AQI, and AQI status.

Power Supply

The prototype is powered using a rechargeable Li-Po battery. A TP4056 module is used for battery charging, while an MT3608 boost converter provides the required voltage for the prototype.

ESPHome and IoT

ESPHome is used to configure the ESP32, connect the sensors, and provide Wi-Fi-based monitoring. The system can be integrated with Home Assistant for dashboard visualization and also provides an ESPHome web interface that can be accessed from a device connected to the same Wi-Fi network.

Communication

The prototype uses two main communication interfaces:

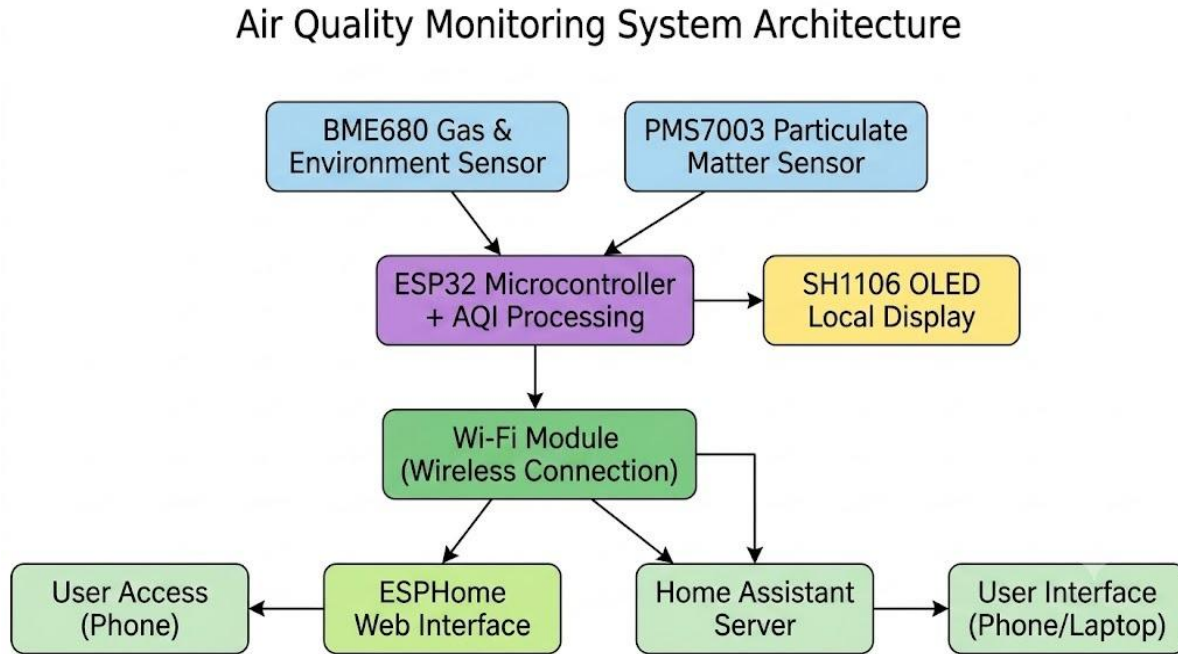
- I²C – BME680 and OLED
- UART – PMS7003

Overall: BME680 + PMS7003 → ESP32 → OLED / Wi-Fi → Home Assistant & Web Interface

5. SYSTEM ARCHITECTURE AND DESIGN

The system is divided into sensing, processing, presentation, communication and power sections.

Figure 1. Overall System Architecture



Sensing:

The **BME680** measures temperature and humidity, while the **PMS7003** measures PM1.0, PM2.5, and PM10. BME680 uses **I²C**, while PMS7003 uses **UART**.

Processing:

The **ESP32** collects the sensor data, processes it, and calculates the PM2.5 and PM10 AQI values.

Display:

The **SH1106 OLED** displays temperature, humidity, PM2.5, PM10, AQI, and AQI status.

Communication:

Wi-Fi enables monitoring through the **ESPHome Web interface** and **Home Assistant**.

Power:

A **Li-Po battery**, **TP4056**, and **MT3608** provide rechargeable and regulated power.

Overall flow:

Sensors → ESP32 → AQI Processing → OLED / Wi-Fi

6. HARDWARE IMPLEMENTATION

The prototype is assembled on a 2.54 mm perfboard. The ESP32 is mounted using female headers so it remains removable. The power modules are grouped near the board edge, while the BME680 is positioned toward the outer edge with the sensing element exposed to ambient air.

Device	Signal	ESP32 Pin / Supply
BME680	SDA	GPIO21
BME680	SCL	GPIO22
BME680	VCC	3.3 V
BME680	GND	GND
OLED	SDA	GPIO21
OLED	SCL	GPIO22
OLED	Address	0x3C
BME680	Address	0x76
PMS7003	TX/RX	GPIO16/17
PMS7003	Baud	9600

Power Architecture

3.7 V LiPo Battery → TP4056 Charging/Protection → MT3608 Boost → Regulated 5 V Supply

The battery connects through a removable connector. The **TP4056** handles charging/protection, while the **MT3608** boosts the battery voltage to the regulated system supply. The converter output was adjusted to approximately **5.00 V** before powering the ESP32.

Safety and assembly practices included disconnecting the battery before soldering, verifying polarity, checking the **MT3608** output with a multimeter, and testing for shorts before applying power.



7. SOFTWARE AND FIRMWARE IMPLEMENTATION

The firmware architecture was implemented using ESPHome with the ESP-IDF framework. The major software blocks are shown below.

Firmware Block	Implementation
Microcontroller	ESP32, ESP-IDF
I ² C	GPIO21 SDA, GPIO22 SCL, 400 kHz
BME680	Address 0x76, environmental readings
UART	GPIO17/16, 9600 baud
PMS7003	PM2.5 and PM10 extraction
AQI	Template/calculation logic
OLED	SH1106, 128×64, 0x3C
Wi-Fi	Configured network + fallback AP
Home Assistant	API integration
Web Server	ESPHome Web interface
OTA	Wireless firmware update
Captive Portal	Network reconfiguration

BME680 update interval was documented as 15 seconds, while PMS7003 values were configured with a 30-second update interval. The OLED refresh was configured at approximately one second.

The firmware also uses secrets for Wi-Fi credentials and Home Assistant encryption-related configuration. This keeps credentials separate from the main YAML configuration.

OTA allows firmware changes to be uploaded through Wi-Fi after initial device setup. It is a firmware-management feature and does not alter the sensor acquisition mechanism.

8. AQI CALCULATION AND DATA PROCESSING

The current AQI implementation uses PM2.5 and PM10 independently. The BME680 temperature and humidity values are not used in the particulate AQI calculation.

PM2.5 Concentration ($\mu\text{g}/\text{m}^3$)	AQI Range
0–30	0–50
31–60	51–100
61–90	101–200
91–120	201–300
121–250	301–400
>250	401–500

PM10 Concentration ($\mu\text{g}/\text{m}^3$)	AQI Range
0–50	0–50
51–100	51–100
101–250	101–200
251–350	201–300
351–430	301–400
>430	401–500

For a measured concentration C within a breakpoint interval, linear interpolation is used:

$$\text{AQI} = [(\text{I high} - \text{I low}) / (\text{C high} - \text{C low})] \times (\text{C} - \text{C low}) + \text{I low}$$

The final AQI is the higher valid pollutant sub-index. For example, if the PM2.5 sub-index is 35 and the PM10 sub-index is 23, the final AQI is 35.

The configured status categories are: 0–25 EXCELLENT, 26–50 GOOD, 51–100 SATISFACTORY, 101–200 MODERATE, 201–300 POOR, 301–400 VERY POOR, and above 400 SEVERE.

9. USER INTERFACES AND IoT CONNECTIVITY

Local OLED Interface

Display Line	Parameter
Line 1	Temperature
Line 2	Humidity
Line 3	AQI
Line 4	AQI status / scanning state



Home Assistant Interface:

The Air Quality Monitor dashboard was designed around a glanceable main AQI/status area, followed by PM2.5 and PM10, environmental readings, and historical trends. The documented entities include AQI, AQI Status, PM2.5, PM10, PM2.5 AQI sub-index, PM10 AQI sub-index, temperature, and humidity.

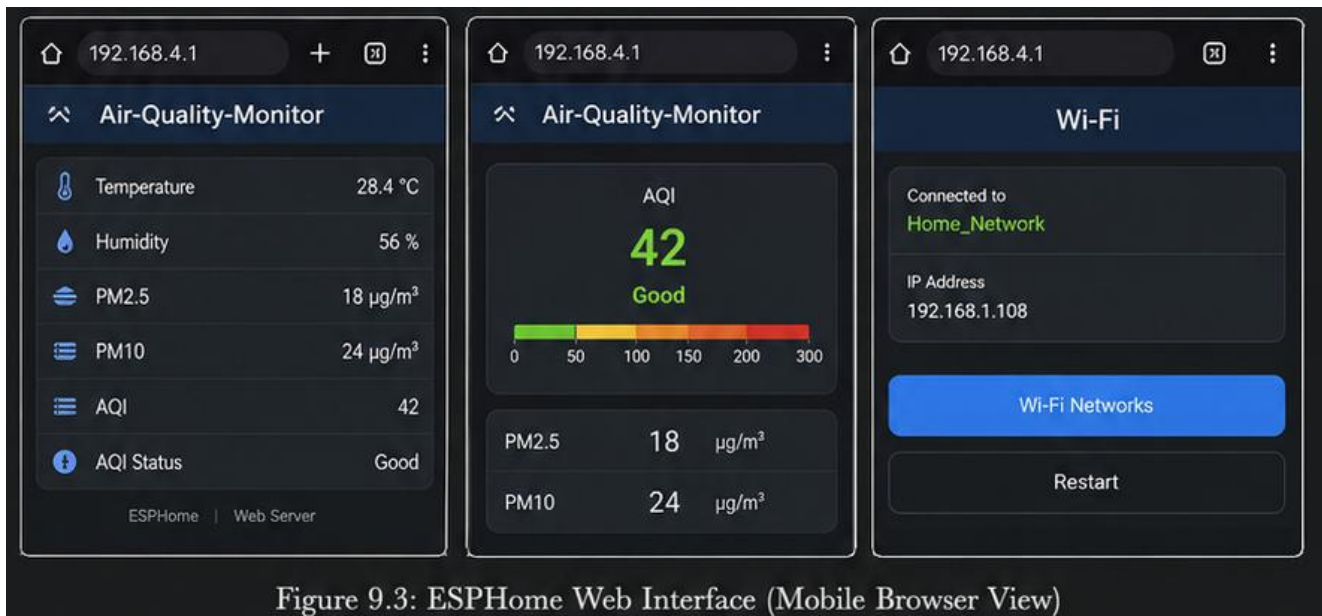
Portable ESPHome Web Interface:

The ESPHome web server provides direct access from a phone browser. This mode is useful for demonstrations because the physical prototype can be taken to another Wi-Fi network without carrying the Home Assistant server.

Fallback Network:

If the configured network is unavailable, the device can expose “Air-Quality-Monitor Fallback” and use a captive portal at the ESP32 access-point address for selecting another Wi-Fi network.

The architecture therefore separates a full IoT dashboard mode from a lightweight portable demonstration mode.



10. METHODOLOGY AND PROTOTYPE DEVELOPMENT

The prototype was developed incrementally so that each hardware and software stage could be tested before proceeding to the next stage.

10.1 Hardware Preparation

1. Defined the concept and requirement for portable air-quality monitoring.
2. Selected the **ESP32, BME680, PMS7003, OLED, and power components**.
3. Planned the perfboard layout and component placement.
4. Installed female headers for removable ESP32 mounting.
5. Soldered and tested the ESP32 socket for electrical continuity.
6. Installed the **TP4056** and **MT3608** power modules.
7. Installed the rechargeable battery connector.
8. Configured the MT3608 output to approximately **5.0 V**.
9. Verified ESP32 power operation using the battery supply.

10.2 Sensor and Display Integration

10. Installed the **BME680** with its sensing element exposed to the surrounding environment.
11. Integrated the **PMS7003** for particulate measurement.
12. Connected and configured the **OLED display** for local monitoring.
13. Tested the sensor readings and display output.

10.3 Firmware and AQI Processing

14. Configured the device using **ESPHome**.
15. Implemented **PM2.5 and PM10 AQI processing**.
16. Configured the OLED to display sensor readings, AQI, and AQI status.

10.4 IoT and Wireless Integration

17. Integrated the device with **Home Assistant**.
18. Configured the **ESPHome Web Server** for browser-based monitoring.
19. Tested the device using different Wi-Fi networks for portable operation.

10.5 Testing and Validation

20. Performed subsystem-level testing and troubleshooting.
21. Used **continuity checks, voltage measurements, I²C scanning, sensor logs, and Wi-Fi logs** to identify hardware and communication issues.
22. Validated the complete system after integrating the individual subsystems.

11. TESTING, VALIDATION AND TROUBLESHOOTING

Testing Results

Test	Observation	Result
ESP32 header continuity	Continuity confirmed; no tested adjacent-pin shorts	PASS
MT3608 output	Adjusted to approximately 5.00 V	PASS
Battery-powered ESP32	ESP32 booted from regulated battery system	PASS
I ² C scan	OLED 0x3C and BME680 0x76 detected after correction	PASS
BME680 readings	Temperature, humidity, pressure and gas resistance produced	PASS
PMS7003	Valid particulate readings obtained after startup	PASS
Home Assistant	Live AQI and sensor entities received	PASS
Alternate Wi-Fi	ESP32 obtained IP on another home Wi-Fi	PASS
Phone hotspot	Some tests produced “No networks found”	REQUIRES OPTIMIZATION
Battery OLED stability	Flicker/reboot observed under battery operation	REQUIRES OPTIMIZATION

I²C Fault Analysis

An intermittent I²C fault caused SDA/SCL errors and prevented device detection. Measurements showed GPIO21 behaved normally when the devices were disconnected but dropped when the BME680 was connected. The issue was traced to a loose/hanging BME680 SDA wire. Correcting the connection restored the I²C bus and both devices were detected.

Power Fault Analysis

The battery-powered OLED showed flickering and the ESP32 later rebooted. Possible causes include battery voltage variation, converter limitations, PMS7003 current demand, wiring resistance, voltage dips, I²C instability, or watchdog/brownout events. The documented approach is to diagnose these separately and measure voltages while the fault is actually occurring.

12. RESULTS, DISCUSSION, LIMITATIONS AND FUTURE SCOPE

Overall Results

System Function	Status
Environmental sensing	Demonstrated
Particulate sensing	Demonstrated
AQI calculation	Implemented
OLED output	Demonstrated
Home Assistant integration	Demonstrated
ESPHome Web	Implemented
Fallback AP/captive portal	Implemented
OTA	Implemented
Portable Wi-Fi	Demonstrated on alternate Wi-Fi; hotspot compatibility remains
Battery stability	Further optimization required

Discussion

The prototype demonstrates successful integration of multiple embedded subsystems. The most important achievement is that the same ESP32 device can provide both local and networked monitoring. The Home Assistant interface is suitable for detailed visualization and future automation, while ESPHome Web reduces the infrastructure required during demonstrations.

Limitations

- The current AQI uses particulate measurements only.
- Home Assistant is currently hosted in VirtualBox on the development laptop and is unavailable when that laptop is off.
- Phone hotspot compatibility requires suitable Wi-Fi configuration.
- Battery-powered OLED flicker/reboot requires additional power-system optimization.
- The prototype is a perfboard implementation rather than a custom PCB.
- The system should be treated as a prototype monitoring device rather than a documented regulatory instrument.

Future Scope

- Custom PCB design and compact enclosure.
- Improved power regulation and battery monitoring.
- Long-term data logging and trend analysis.
- Mobile application and remote alerts.
- Calibration and comparison with reference instruments.
- Improved airflow and sensor placement.
- Always-on Home Assistant host for 24/7 monitoring.
- Expanded environmental sensing and advanced analytics.

13. CONCLUSION

Conclusion

The Portable ESP32-Based Air Quality Monitoring System successfully demonstrates a compact IoT architecture for environmental and particulate monitoring. The system integrates BME680 environmental sensing, PMS7003 particulate sensing, ESP32 processing, AQI calculation, OLED visualization, rechargeable battery power, Wi-Fi, Home Assistant, and ESPHome Web.

The project also demonstrates the practical engineering process involved in building an embedded prototype. Hardware faults such as the I²C SDA connection problem were isolated using electrical measurements and corrected. Wireless testing confirmed operation on an alternate Wi-Fi network, while the remaining battery-stability and hotspot-compatibility issues were identified as clear areas for future optimization.

The final architecture provides a useful balance between local operation and IoT connectivity, making the prototype suitable for technical demonstration and as a foundation for future development.

References / Technical Resources

1. Project development notes and prototype testing record supplied for this report.
2. Espressif ESP32 technical documentation and ESP-IDF documentation.
3. Bosch Sensortec BME680 technical documentation/datasheet.
4. Plantower PMS7003 particulate matter sensor documentation.
5. SH1106 OLED controller/module documentation.
6. ESPHome documentation and component references.
7. Home Assistant documentation and integration references.
8. TP4056 charger/protection module documentation.
9. MT3608 DC-DC boost converter module documentation.

-x-x-x- End of Report -x-x-x-